

Définitions

- Un **algorithme** est une suite d'instructions et d'opérations permettant de résoudre un problème.

Définitions

- Un **algorithme** est une suite d'instructions et d'opérations permettant de résoudre un problème.

Par exemple pour résoudre le problème du tri d'une liste de valeurs, on peut utiliser l'algorithme du tri par insertion. Cela consiste à prendre une à une chaque valeur et à l'insérer au bon emplacement dans une liste initialement vide.

Définitions

- Un **algorithme** est une suite d'instructions et d'opérations permettant de résoudre un problème.
Par exemple pour résoudre le problème du tri d'une liste de valeurs, on peut utiliser l'algorithme du tri par insertion. Cela consiste à prendre une à une chaque valeur et à l'insérer au bon emplacement dans une liste initialement vide.
- Un **programme** est la traduction d'un algorithme dans un langage informatique. On dit qu'un programme est la mise en œuvre d'un algorithme.

Définitions

- Un **algorithme** est une suite d'instructions et d'opérations permettant de résoudre un problème.
Par exemple pour résoudre le problème du tri d'une liste de valeurs, on peut utiliser l'algorithme du tri par insertion. Cela consiste à prendre une à une chaque valeur et à l'insérer au bon emplacement dans une liste initialement vide.
- Un **programme** est la traduction d'un algorithme dans un langage informatique. On dit qu'un programme est la mise en œuvre d'un algorithme. L'algorithme du tri par insertion peut être écrit en Python, en C, ...

Définitions

- Un **algorithme** est une suite d'instructions et d'opérations permettant de résoudre un problème.
Par exemple pour résoudre le problème du tri d'une liste de valeurs, on peut utiliser l'algorithme du tri par insertion. Cela consiste à prendre une à une chaque valeur et à l'insérer au bon emplacement dans une liste initialement vide.
- Un **programme** est la traduction d'un algorithme dans un langage informatique. On dit qu'un programme est la mise en œuvre d'un algorithme. L'algorithme du tri par insertion peut être écrit en Python, en C, ...
- Un **paradigme** de programmation est une façon d'approcher un problème, d'en concevoir et d'en modéliser une solution.

Définitions

- Un **algorithme** est une suite d'instructions et d'opérations permettant de résoudre un problème.
Par exemple pour résoudre le problème du tri d'une liste de valeurs, on peut utiliser l'algorithme du tri par insertion. Cela consiste à prendre une à une chaque valeur et à l'insérer au bon emplacement dans une liste initialement vide.
- Un **programme** est la traduction d'un algorithme dans un langage informatique. On dit qu'un programme est la mise en œuvre d'un algorithme. L'algorithme du tri par insertion peut être écrit en Python, en C, ...
- Un **paradigme** de programmation est une façon d'approcher un problème, d'en concevoir et d'en modéliser une solution.
Le langage C est une illustration du paradigme de programmation impérative. Le langage OCaml nous permettra d'illustrer le paradigme fonctionnel.

Exemple introductif

On considère le programme C suivant :

```
1  int get_imax(int tab[],int size) {  
2      int max = 0;  
3      int imax;  
4      for (int i=0; i<size; i++) {  
5          if (tab[i] >= max) {  
6              max = tab[i];  
7              imax = i;}}  
8      return imax;}
```

- ❶ Quel est le résultat renvoyé si le tableau fourni en argument contient dans cet ordre les valeurs : 12, 18, 11, 9, 10?

Exemple introductif

On considère le programme C suivant :

```
1  int get_imax(int tab[],int size) {  
2      int max = 0;  
3      int imax;  
4      for (int i=0; i<size; i++) {  
5          if (tab[i] >= max) {  
6              max = tab[i];  
7              imax = i;}}  
8      return imax;}
```

- 1 Quel est le résultat renvoyé si le tableau fourni en argument contient dans cet ordre les valeurs : 12, 18, 11, 9, 10?
- 2 Même question avec le tableau 12, 18, 11, 18, 10.

Exemple introductif

On considère le programme C suivant :

```
1  int get_imax(int tab[],int size) {  
2      int max = 0;  
3      int imax;  
4      for (int i=0; i<size; i++) {  
5          if (tab[i] >= max) {  
6              max = tab[i];  
7              imax = i;}}  
8      return imax;}
```

- 1 Quel est le résultat renvoyé si le tableau fourni en argument contient dans cet ordre les valeurs : 12, 18, 11, 9, 10?
- 2 Même question avec le tableau 12, 18, 11, 18, 10.
- 3 Même question avec le tableau -12, -15, -7.

Exemple introductif

On considère le programme C suivant :

```
1  int get_imax(int tab[],int size) {  
2      int max = 0;  
3      int imax;  
4      for (int i=0; i<size; i++) {  
5          if (tab[i] >= max) {  
6              max = tab[i];  
7              imax = i;}}  
8      return imax;}
```

- 1 Quel est le résultat renvoyé si le tableau fourni en argument contient dans cet ordre les valeurs : 12, 18, 11, 9, 10?
- 2 Même question avec le tableau 12, 18, 11, 18, 10.
- 3 Même question avec le tableau -12, -15, -7.
- 4 Même question si le tableau est vide (c'est-à-dire que `size == 0`).

Correction

- ① La fonction renvoie 1 (indice de l'élément maximal du tableau).

Définition

Correction

- ① La fonction renvoie 1 (indice de l'élément maximal du tableau).
- ② La fonction renvoie 3, c'est l'indice de la dernière occurrence du maximum des éléments du tableau.

Définition

Correction

- ① La fonction renvoie 1 (indice de l'élément maximal du tableau).
- ② La fonction renvoie 3, c'est l'indice de la dernière occurrence du maximum des éléments du tableau.
- ③ C'est un comportement indéfini, la variable `imax` n'est pas initialisée.

Définition

Correction

- ❶ La fonction renvoie 1 (indice de l'élément maximal du tableau).
- ❷ La fonction renvoie 3, c'est l'indice de la dernière occurrence du maximum des éléments du tableau.
- ❸ C'est un comportement indéfini, la variable `imax` n'est pas initialisée.
- ❹ Une nouvelle fois, le comportement est indéfini car on renvoie la variable `imax` qui n'a jamais été initialisée.

Définition

Correction

- ❶ La fonction renvoie 1 (indice de l'élément maximal du tableau).
- ❷ La fonction renvoie 3, c'est l'indice de la dernière occurrence du maximum des éléments du tableau.
- ❸ C'est un comportement indéfini, la variable `imax` n'est pas initialisée.
- ❹ Une nouvelle fois, le comportement est indéfini car on renvoie la variable `imax` qui n'a jamais été initialisée.

Définition

Écrire la **spécification** d'une fonction, c'est donner une description formelle et détaillée de ses caractéristiques. En particulier :

Correction

- ❶ La fonction renvoie 1 (indice de l'élément maximal du tableau).
- ❷ La fonction renvoie 3, c'est l'indice de la dernière occurrence du maximum des éléments du tableau.
- ❸ C'est un comportement indéfini, la variable `imax` n'est pas initialisée.
- ❹ Une nouvelle fois, le comportement est indéfini car on renvoie la variable `imax` qui n'a jamais été initialisée.

Définition

Écrire la **spécification** d'une fonction, c'est donner une description formelle et détaillée de ses caractéristiques. En particulier :

- les entrées admissibles : types et valeurs possibles des arguments (**préconditions**),

Correction

- ❶ La fonction renvoie 1 (indice de l'élément maximal du tableau).
- ❷ La fonction renvoie 3, c'est l'indice de la dernière occurrence du maximum des éléments du tableau.
- ❸ C'est un comportement indéfini, la variable `imax` n'est pas initialisée.
- ❹ Une nouvelle fois, le comportement est indéfini car on renvoie la variable `imax` qui n'a jamais été initialisée.

Définition

Écrire la **spécification** d'une fonction, c'est donner une description formelle et détaillée de ses caractéristiques. En particulier :

- les entrées admissibles : types et valeurs possibles des arguments (**préconditions**),
- ce que renvoie la fonction et les effets de bord (*side effects*) éventuels : modification des arguments, affichage, ... (ce sont les **postconditions**).

Remarques

- La spécification est souvent donnée en commentaire dans le code source.

Remarques

- La spécification est souvent donnée en commentaire dans le code source.
Les commentaires en C s'écrivent entre `/*` et `*/` et en OCaml entre `(*` et `*)`.

Exemples

Remarques

- La spécification est souvent donnée en commentaire dans le code source. Les commentaires en C s'écrivent entre `/*` et `*/` et en OCaml entre `(*` et `*)`.
- La vérification des préconditions peut s'effectuer à l'aide d'instructions `assert`. Elles sont de la forme `assert(condition)` en C, et nécessitent d'importer `assert.h`. Si la condition échoue, le programme s'arrête et affiche une erreur, c'est ce qu'on appelle de la programmation défensive (anticipation des erreurs).

Exemples

Remarques

- La spécification est souvent donnée en commentaire dans le code source. Les commentaires en C s'écrivent entre `/*` et `*/` et en OCaml entre `(*` et `*)`.
- La vérification des préconditions peut s'effectuer à l'aide d'instructions `assert`. Elles sont de la forme `assert(condition)` en C, et nécessitent d'importer `assert.h`. Si la condition échoue, le programme s'arrête et affiche une erreur, c'est ce qu'on appelle de la programmation défensive (anticipation des erreurs).

Exemples

Écrire en C une fonction qui :

Remarques

- La spécification est souvent donnée en commentaire dans le code source. Les commentaires en C s'écrivent entre `/*` et `*/` et en OCaml entre `(*` et `*)`.
- La vérification des préconditions peut s'effectuer à l'aide d'instructions `assert`. Elles sont de la forme `assert(condition)` en C, et nécessitent d'importer `assert.h`. Si la condition échoue, le programme s'arrête et affiche une erreur, c'est ce qu'on appelle de la programmation défensive (anticipation des erreurs).

Exemples

Écrire en C une fonction qui :

- Accepte en argument un tableau *non vide* d'entiers.

Remarques

- La spécification est souvent donnée en commentaire dans le code source. Les commentaires en C s'écrivent entre `/*` et `*/` et en OCaml entre `(*` et `*)`.
- La vérification des préconditions peut s'effectuer à l'aide d'instructions `assert`. Elles sont de la forme `assert(condition)` en C, et nécessitent d'importer `assert.h`. Si la condition échoue, le programme s'arrête et affiche une erreur, c'est ce qu'on appelle de la programmation défensive (anticipation des erreurs).

Exemples

Écrire en C une fonction qui :

- Accepte en argument un tableau *non vide* d'entiers.
- Renvoie l'indice de la première occurrence du maximum des éléments de ce tableau.

Correction

```
1  #include <assert.h>
2
3  int get_imax(int tab[],int size) {
4      /* Prend en argument un tableau non vide d'entiers
5       Renvoie l'indice de la 1ere occurrence du maximum du tableau*/
6      assert (size > 0);
7      int max = tab[0];
8      int imax = 0;
9      for (int i=0; i<size; i++) {
10         if (tab[i] > max) {
11             max = tab[i];
12             imax = i;}}
13     return imax;}
```


Jeu de tests

Le comportement correct d'une fonction peut être « validé » (mais pas prouvé), par l'utilisation d'un **jeu de tests**, c'est-à-dire un ensemble de couples d'entrées du programme et de sorties attendues.

Jeu de tests

Le comportement correct d'une fonction peut être « validé » (mais pas prouvé), par l'utilisation d'un **jeu de tests**, c'est-à-dire un ensemble de couples d'entrées du programme et de sorties attendues.

Dans la fonction précédente, on pourrait tester (entre autres) des cas limites (*edge cases*), comme par exemple un tableau à un seul élément, ou vide, ou des situations où le maximum se trouve en première ou dernière position du tableau.

Exemple

Jeu de tests

Le comportement correct d'une fonction peut être « validé » (mais pas prouvé), par l'utilisation d'un **jeu de tests**, c'est-à-dire un ensemble de couples d'entrées du programme et de sorties attendues.

Dans la fonction précédente, on pourrait tester (entre autres) des cas limites (*edge cases*), comme par exemple un tableau à un seul élément, ou vide, ou des situations où le maximum se trouve en première ou dernière position du tableau.

Exemple

- ❶ Écrire une fonction `contient_double` qui prend en argument un tableau d'entiers et renvoie `true` si ce tableau contient deux éléments consécutifs égaux.

Jeu de tests

Le comportement correct d'une fonction peut être « validé » (mais pas prouvé), par l'utilisation d'un **jeu de tests**, c'est-à-dire un ensemble de couples d'entrées du programme et de sorties attendues.

Dans la fonction précédente, on pourrait tester (entre autres) des cas limites (*edge cases*), comme par exemple un tableau à un seul élément, ou vide, ou des situations où le maximum se trouve en première ou dernière position du tableau.

Exemple

- 1 Écrire une fonction `contient_double` qui prend en argument un tableau d'entiers et renvoie `true` si ce tableau contient deux éléments consécutifs égaux.
- 2 Proposer un jeu de tests pour cette fonction.

Correction

1

```
bool contient_double(int tab[],int size){  
2     for (int i=0;i<size-1;i++){  
3         if (tab[i]==tab[i+1]){  
4             return true;}}  
5     return false;}
```

Correction

1

```
1  bool contient_double(int tab[],int size){  
2      for (int i=0;i<size-1;i++){  
3          if (tab[i]==tab[i+1]){  
4              return true;}}  
5      return false;}
```

2 On pourrait tester les cas suivants :

Correction

```
1  bool contient_double(int tab[],int size){  
2      for (int i=0;i<size-1;i++){  
3          if (tab[i]==tab[i+1]){  
4              return true;}}  
5      return false;}
```

- 2 On pourrait tester les cas suivants :
- Tableau vide ou à un seul élément ;

Correction

1

```
bool contient_double(int tab[],int size){  
2     for (int i=0;i<size-1;i++){  
3         if (tab[i]==tab[i+1]){  
4             return true;}}  
5     return false;}
```

- 2 On pourrait tester les cas suivants :
- Tableau vide ou à un seul élément ;
 - Mêmes éléments mais non consécutifs ;

Correction

1

```
bool contient_double(int tab[],int size){  
2     for (int i=0;i<size-1;i++){  
3         if (tab[i]==tab[i+1]){  
4             return true;}}  
5     return false;}
```

- 2 On pourrait tester les cas suivants :
- Tableau vide ou à un seul élément ;
 - Mêmes éléments mais non consécutifs ;
 - Élément présent en plus de deux exemplaires consécutifs ;

Correction

1

```
bool contient_double(int tab[],int size){  
2     for (int i=0;i<size-1;i++){  
3         if (tab[i]==tab[i+1]){  
4             return true;}}  
5     return false;}
```

- 2 On pourrait tester les cas suivants :
- Tableau vide ou à un seul élément ;
 - Mêmes éléments mais non consécutifs ;
 - Élément présent en plus de deux exemplaires consécutifs ;
 - Présence du double en tout début ou toute fin de tableau.

Partitionnement du domaine d'entrée

- Pour concevoir un jeu de tests représentatif, on **partitionne** le domaine des données d'entrée en **classes d'équivalence** (sous-ensembles d'entrées pour lesquels le programme est censé adopter le même comportement).

Partitionnement du domaine d'entrée

- Pour concevoir un jeu de tests représentatif, on **partitionne** le domaine des données d'entrée en **classes d'équivalence** (sous-ensembles d'entrées pour lesquels le programme est censé adopter le même comportement).
- On sélectionne au moins une valeur représentative par classe d'équivalence.

Partitionnement du domaine d'entrée

- Pour concevoir un jeu de tests représentatif, on **partitionne** le domaine des données d'entrée en **classes d'équivalence** (sous-ensembles d'entrées pour lesquels le programme est censé adopter le même comportement).
- On sélectionne au moins une valeur représentative par classe d'équivalence.
- On teste systématiquement les **valeurs limites** (*edge cases* ou frontières) : cas extrêmes, entrées nulles, tableaux vides ou à un élément, début/fin de structure, etc.

Exemple introductif

Écrire la fonction `duree_vol` qui prend en argument un entier positif n et renvoie le nombre d'itérations nécessaires avant d'atteindre 1 en prenant cet entier comme valeur initiale dans la suite de Syracuse. On rappelle que :

$$s_{n+1} = \begin{cases} \frac{s_n}{2} & \text{si } n \text{ est pair} \\ 3s_n + 1 & \text{sinon} \end{cases}$$

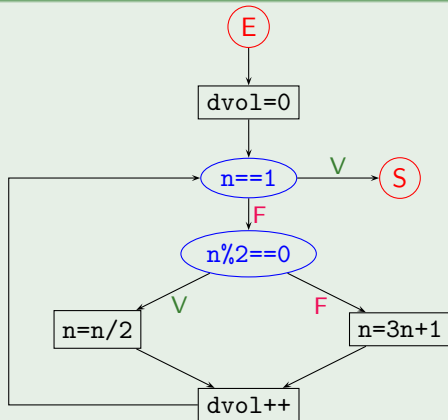
Exemple introductif

Écrire la fonction `duree_vol` qui prend en argument un entier positif n et renvoie le nombre d'itérations nécessaires avant d'atteindre 1 en prenant cet entier comme valeur initiale dans la suite de Syracuse. On rappelle que :

$$s_{n+1} = \begin{cases} \frac{s_n}{2} & \text{si } n \text{ est pair} \\ 3s_n + 1 & \text{sinon} \end{cases}$$

```
1  int duree_vol(int n) {  
2      int dvol = 0;  
3      while (n!=1) {  
4          if (n%2==0)  
5              {n = n/2;}  
6          else  
7              {n = 3*n+1;}  
8          dvol ++;}  
9      return dvol;}  
10
```

Représentation des exécutions possibles



Graphe de flot de contrôle

Le **graphe de flot de contrôle** représente les exécutions possibles d'un programme.

- Les sommets **E** et **S** représentent l'entrée et la sortie ;

Graphe de flot de contrôle

Le **graphe de flot de contrôle** représente les exécutions possibles d'un programme.

- Les sommets **E** et **S** représentent l'entrée et la sortie ;
- Les autres nœuds sont les blocs d'instructions ;

Graphe de flot de contrôle

Le **graphe de flot de contrôle** représente les exécutions possibles d'un programme.

- Les sommets **E** et **S** représentent l'entrée et la sortie ;
- Les autres nœuds sont les blocs d'instructions ;
- On dessine un arc entre deux nœuds A et B lorsque l'exécution de B peut suivre celle de A ;

Graphe de flot de contrôle

Le **graphe de flot de contrôle** représente les exécutions possibles d'un programme.

- Les sommets **E** et **S** représentent l'entrée et la sortie ;
- Les autres nœuds sont les blocs d'instructions ;
- On dessine un arc entre deux nœuds A et B lorsque l'exécution de B peut suivre celle de A ;
- Les arcs quittant les instructions conditionnelles sont étiquetés par V ou F ;

Graphe de flot de contrôle

Le **graphe de flot de contrôle** représente les exécutions possibles d'un programme.

- Les sommets **E** et **S** représentent l'entrée et la sortie ;
- Les autres nœuds sont les blocs d'instructions ;
- On dessine un arc entre deux nœuds A et B lorsque l'exécution de B peut suivre celle de A ;
- Les arcs quittant les instructions conditionnelles sont étiquetés par V ou F ;
- On dira qu'un chemin dans le graphe est **faisable** lorsqu'il existe des valeurs d'entrées pour lesquelles l'exécution passe par tous les nœuds de ce chemin.

Critères de couverture

Un jeu de tests peut viser plusieurs niveaux de couverture :

- **Couverture des sommets (instructions)** : passer au moins une fois par chaque nœud d'instruction.

Critères de couverture

Un jeu de tests peut viser plusieurs niveaux de couverture :

- **Couverture des sommets (instructions)** : passer au moins une fois par chaque nœud d'instruction.
- **Couverture des arcs (branches)** : emprunter au moins une fois chaque arc (teste chaque issue des branchements conditionnels).

Critères de couverture

Un jeu de tests peut viser plusieurs niveaux de couverture :

- **Couverture des sommets (instructions)** : passer au moins une fois par chaque nœud d'instruction.
- **Couverture des arcs (branches)** : emprunter au moins une fois chaque arc (teste chaque issue des branchements conditionnels).
- **Couverture des chemins** : parcourir tous les chemins d'exécution possibles. En présence de boucles, le nombre de chemins est potentiellement infini : on se limite en pratique aux *chemins sans cycle* ou à un nombre borné d'itérations.

Exemple

- 1 Montrer que le jeu de tests $\{n=1, n=8\}$ ne couvre ni tous les arcs, ni tous les sommets.

Exemple

- 1 Montrer que le jeu de tests $\{n=1, n=8\}$ ne couvre ni tous les arcs, ni tous les sommets.

On ne passe jamais par le nœud représentant l'instruction $n=3*n+1$, en effet pour $n=1$ on atteint directement la sortie S et pour $n=8$, toutes les valeurs de la suite sont paires.

Exemple

- 1 Montrer que le jeu de tests $\{n=1, n=8\}$ ne couvre ni tous les arcs, ni tous les sommets.

On ne passe jamais par le nœud représentant l'instruction $n=3*n+1$, en effet pour $n=1$ on atteint directement la sortie S et pour $n=8$, toutes les valeurs de la suite sont paires.

- 2 Proposer un jeu de tests permettant de couvrir tous les arcs.

Exemple

- 1 Montrer que le jeu de tests $\{n=1, n=8\}$ ne couvre ni tous les arcs, ni tous les sommets.

On ne passe jamais par le nœud représentant l'instruction $n=3*n+1$, en effet pour $n=1$ on atteint directement la sortie S et pour $n=8$, toutes les valeurs de la suite sont paires.

- 2 Proposer un jeu de tests permettant de couvrir tous les arcs.

On peut ajouter par exemple le test $n=5$.

Exercice

Exemple

- 1 Montrer que le jeu de tests $\{n=1, n=8\}$ ne couvre ni tous les arcs, ni tous les sommets.

On ne passe jamais par le nœud représentant l'instruction $n=3*n+1$, en effet pour $n=1$ on atteint directement la sortie S et pour $n=8$, toutes les valeurs de la suite sont paires.

- 2 Proposer un jeu de tests permettant de couvrir tous les arcs.

On peut ajouter par exemple le test $n=5$.

Exercice

- 1 Écrire une fonction prenant en argument un flottant x et un entier positif ou nul n et qui renvoie x puissance n .

Exemple

- 1 Montrer que le jeu de tests $\{n=1, n=8\}$ ne couvre ni tous les arcs, ni tous les sommets.

On ne passe jamais par le nœud représentant l'instruction $n=3*n+1$, en effet pour $n=1$ on atteint directement la sortie S et pour $n=8$, toutes les valeurs de la suite sont paires.

- 2 Proposer un jeu de tests permettant de couvrir tous les arcs.

On peut ajouter par exemple le test $n=5$.

Exercice

- 1 Écrire une fonction prenant en argument un flottant x et un entier positif ou nul n et qui renvoie x puissance n .
- 2 Tracer son graphe de flot de contrôle.

Exemple

- 1 Montrer que le jeu de tests $\{n=1, n=8\}$ ne couvre ni tous les arcs, ni tous les sommets.

On ne passe jamais par le nœud représentant l'instruction $n=3*n+1$, en effet pour $n=1$ on atteint directement la sortie S et pour $n=8$, toutes les valeurs de la suite sont paires.

- 2 Proposer un jeu de tests permettant de couvrir tous les arcs.

On peut ajouter par exemple le test $n=5$.

Exercice

- 1 Écrire une fonction prenant en argument un flottant x et un entier positif ou nul n et qui renvoie x puissance n .
- 2 Tracer son graphe de flot de contrôle.
- 3 Proposer un jeu de tests.

Correction

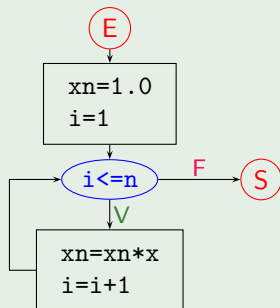
```
1  double puissance(double x, int n){  
2      assert (n>=0);  
3      double xn = 1.0;  
4      for (int i=1;i<=n;i++) {  
5          xn = xn * x;}  
6      return xn;}  

```


Correction

```
1  double puissance(double x, int n){  
2      assert (n>=0);  
3      double xn = 1.0;  
4      for (int i=1;i<=n;i++) {  
5          xn = xn * x;}  
6      return xn;}  

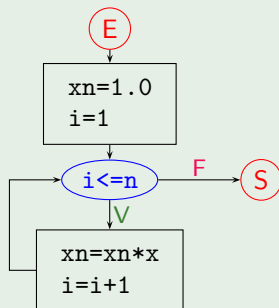
```



Correction

```
1 double puissance(double x, int n){  
2     assert (n>=0);  
3     double xn = 1.0;  
4     for (int i=1;i<=n;i++) {  
5         xn = xn * x;}  
6     return xn;}  

```

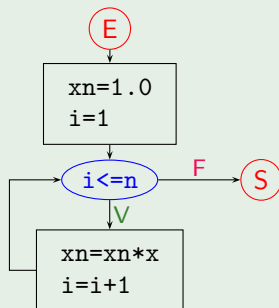


On peut proposer le jeu de tests suivant :

- `x=2.0, n=0`

Correction

```
1  double puissance(double x, int n){  
2      assert (n>=0);  
3      double xn = 1.0;  
4      for (int i=1;i<=n;i++) {  
5          xn = xn * x;}  
6      return xn;}
```



On peut proposer le jeu de tests suivant :

- `x=2.0, n=0`
- `x=2.0, n=3`

Remarque

Lorsqu'une instruction conditionnelle comporte des conjonctions ou des disjonctions, on peut formuler des tests qui prévoient toutes les possibilités de la satisfaire.

Exemple

Une année est bissextile si elle est divisible par 4 mais pas par 100 ou si elle est divisible par 400.

- 1 Écrire une fonction `bissextile` qui prend en argument un entier positif `annee` et renvoie `true` si l'année est bissextile et `false` sinon.

Remarque

Lorsqu'une instruction conditionnelle comporte des conjonctions ou des disjonctions, on peut formuler des tests qui prévoient toutes les possibilités de la satisfaire.

Exemple

Une année est bissextile si elle est divisible par 4 mais pas par 100 ou si elle est divisible par 400.

- 1 Écrire une fonction `bissextile` qui prend en argument un entier positif `annee` et renvoie `true` si l'année est bissextile et `false` sinon.
- 2 Proposer un jeu de tests pour cette fonction qui prévoit toutes les possibilités de satisfaire la condition d'être bissextile.

Correction

1

```
1  bool bissextile(int annee) {  
2      return ((annee % 4 == 0 && annee % 100 != 0) || (annee  
    ↪ % 400 == 0));  
3  }
```

Correction

1

```
1  bool bissextile(int annee) {  
2      return ((annee % 4 == 0 && annee % 100 != 0) || (annee  
    ↪ % 400 == 0));  
3  }
```

2

On peut proposer les tests suivants :

Correction

1

```
1  bool bissextile(int annee) {  
2      return ((annee % 4 == 0 && annee % 100 != 0) || (annee  
    ↪ % 400 == 0));  
3  }
```

2

On peut proposer les tests suivants :

- `annee = 2004` qui permet de valider la première condition ;

Correction

1

```
1  bool bissextile(int annee) {  
2      return ((annee % 4 == 0 && annee % 100 != 0) || (annee  
    ↪ % 400 == 0));  
3  }
```

2

On peut proposer les tests suivants :

- `annee = 2004` qui permet de valider la première condition ;
- `annee = 2000` qui permet de valider la deuxième condition.

Exemple introductif

Algorithme : Multiplier sans utiliser *

Entrées : $n \in \mathbb{N}, m \in \mathbb{N}$

Sorties : nm

```
1  $r \leftarrow 0$ 
2 tant que  $m > 0$  faire
3    $m \leftarrow m - 1$ 
4    $r \leftarrow r + n$ 
5 fin
6 return  $r$ 
```

Exemple introductif

Algorithme : Multiplier sans utiliser *

Entrées : $n \in \mathbb{N}, m \in \mathbb{N}$

Sorties : nm

```
1  $r \leftarrow 0$ 
2 tant que  $\underline{m > 0}$  faire
3    $m \leftarrow m - 1$ 
4    $r \leftarrow r + n$ 
5 fin
6 return  $r$ 
```

① Donner une implémentation en C de cet algorithme.

Exemple introductif

Algorithme : Multiplier sans utiliser *

Entrées : $n \in \mathbb{N}, m \in \mathbb{N}$

Sorties : nm

```
1  $r \leftarrow 0$ 
2 tant que  $m > 0$  faire
3    $m \leftarrow m - 1$ 
4    $r \leftarrow r + n$ 
5 fin
6 return  $r$ 
```

- ① Donner une implémentation en C de cet algorithme.
- ② Montrer que dans la boucle « tant que » :
 - m ne prend que des valeurs entières ;

Exemple introductif

Algorithme : Multiplier sans utiliser *

Entrées : $n \in \mathbb{N}, m \in \mathbb{N}$

Sorties : nm

```
1  $r \leftarrow 0$ 
2 tant que  $m > 0$  faire
3    $m \leftarrow m - 1$ 
4    $r \leftarrow r + n$ 
5 fin
6 return  $r$ 
```

- ① Donner une implémentation en C de cet algorithme.
- ② Montrer que dans la boucle « tant que » :
 - m ne prend que des valeurs entières ;
 - m prend des valeurs positives ;

Exemple introductif

Algorithme : Multiplier sans utiliser *

Entrées : $n \in \mathbb{N}, m \in \mathbb{N}$

Sorties : nm

```
1  $r \leftarrow 0$ 
2 tant que  $m > 0$  faire
3    $m \leftarrow m - 1$ 
4    $r \leftarrow r + n$ 
5 fin
6 return  $r$ 
```

- ① Donner une implémentation en C de cet algorithme.
- ② Montrer que dans la boucle « tant que » :
 - m ne prend que des valeurs entières ;
 - m prend des valeurs positives ;
 - les valeurs prises par m sont *strictement* décroissantes.

Définitions

Définitions

- On dit qu'un algorithme **termine** lorsqu'il renvoie un résultat en un nombre fini d'étapes quelles que soient les valeurs des entrées.

Définitions

- On dit qu'un algorithme **termine** lorsqu'il renvoie un résultat en un nombre fini d'étapes quelles que soient les valeurs des entrées.
- Un **variant de boucle** est une quantité :

Définitions

- On dit qu'un algorithme **termine** lorsqu'il renvoie un résultat en un nombre fini d'étapes quelles que soient les valeurs des entrées.
- Un **variant de boucle** est une quantité :
 - ① à valeurs entières,

Définitions

- On dit qu'un algorithme **termine** lorsqu'il renvoie un résultat en un nombre fini d'étapes quelles que soient les valeurs des entrées.
- Un **variant de boucle** est une quantité :
 - 1 à valeurs entières,
 - 2 positives,

Définitions

- On dit qu'un algorithme **termine** lorsqu'il renvoie un résultat en un nombre fini d'étapes quelles que soient les valeurs des entrées.
- Un **variant de boucle** est une quantité :
 - 1 à valeurs entières,
 - 2 positives,
 - 3 qui décroît *strictement* à chaque passage dans la boucle.

Définitions

- On dit qu'un algorithme **termine** lorsqu'il renvoie un résultat en un nombre fini d'étapes quelles que soient les valeurs des entrées.
- Un **variant de boucle** est une quantité :
 - 1 à valeurs entières,
 - 2 positives,
 - 3 qui décroît *strictement* à chaque passage dans la boucle.

! Propriété

Si une boucle admet un variant, alors cette boucle termine.

Définitions

- On dit qu'un algorithme **termine** lorsqu'il renvoie un résultat en un nombre fini d'étapes quelles que soient les valeurs des entrées.
- Un **variant de boucle** est une quantité :
 - 1 à valeurs entières,
 - 2 positives,
 - 3 qui décroît *strictement* à chaque passage dans la boucle.

! Propriété

Si une boucle admet un variant, alors cette boucle termine.

Exemple

m est un variant de boucle de l'algorithme de multiplication ci-dessus et donc cet algorithme termine.

Exemple

Algorithme : Nombre de chiffres en base 10

Entrées : $n \in \mathbb{N}$

Sorties : p , nombre de chiffres de n dans son écriture en base 10.

```
1 si  $n = 0$  alors
2   | return 1
3 fin
4  $p \leftarrow 0$ 
5 tant que  $n > 0$  faire
6   |  $p \leftarrow p + 1$ 
7   |  $n \leftarrow \lfloor \frac{n}{10} \rfloor$ 
8 fin
9 return  $p$ 
```

Exemple

Algorithme : Nombre de chiffres en base 10

Entrées : $n \in \mathbb{N}$

Sorties : p , nombre de chiffres de n dans son écriture en base 10.

```
1 si  $n = 0$  alors
2   |   return 1
3 fin
4  $p \leftarrow 0$ 
5 tant que  $n > 0$  faire
6   |    $p \leftarrow p + 1$ 
7   |    $n \leftarrow \lfloor \frac{n}{10} \rfloor$ 
8 fin
9 return  $p$ 
```

① Donner une implémentation en C de cet algorithme.

Exemple

Algorithme : Nombre de chiffres en base 10

Entrées : $n \in \mathbb{N}$

Sorties : p , nombre de chiffres de n dans son écriture en base 10.

```
1 si  $n = 0$  alors
2   |   return 1
3 fin
4  $p \leftarrow 0$ 
5 tant que  $n > 0$  faire
6   |    $p \leftarrow p + 1$ 
7   |    $n \leftarrow \lfloor \frac{n}{10} \rfloor$ 
8 fin
9 return  $p$ 
```

- ① Donner une implémentation en C de cet algorithme.
- ② Prouver la terminaison de cet algorithme.

Implémentation en C

```
1  int nb_chiffres(int n)
2  {
3      assert(n >= 0);
4      int p = 0;
5      if (n == 0)
6      {
7          return 1;
8      }
9      while (n > 0)
10     {
11         n = n / 10;
12         p = p + 1;
13     }
14     return p;
15 }
```

Preuve de terminaison

Montrons que n est un variant de la boucle « tant que » de l'algorithme :

- 1 n ne prend que des valeurs entières, en effet, $n \in \mathbb{N}$ en entrée et $\left\lfloor \frac{n}{10} \right\rfloor$ est entier.

Preuve de terminaison

Montrons que n est un variant de la boucle « tant que » de l'algorithme :

- 1 n ne prend que des valeurs entières, en effet, $n \in \mathbb{N}$ en entrée et $\left\lfloor \frac{n}{10} \right\rfloor$ est entier.
- 2 n est positif par condition d'entrée dans la boucle.

Preuve de terminaison

Montrons que n est un variant de la boucle « tant que » de l'algorithme :

- 1 n ne prend que des valeurs entières, en effet, $n \in \mathbb{N}$ en entrée et $\left\lfloor \frac{n}{10} \right\rfloor$ est entier.
- 2 n est positif par condition d'entrée dans la boucle.
- 3 n décroît strictement à chaque passage dans la boucle car comme $n > 0$, $\left\lfloor \frac{n}{10} \right\rfloor < n$.

Exercice

- 1 Écrire un algorithme qui prend en entrée un entier $n > 1$ et renvoie le premier diviseur strictement supérieur à 1 de cet entier. Par exemple pour $n = 7$ l'algorithme renvoie 7 et pour $n = 15$, l'algorithme renvoie 3.

Exercice

- 1 Écrire un algorithme qui prend en entrée un entier $n > 1$ et renvoie le premier diviseur strictement supérieur à 1 de cet entier. Par exemple pour $n = 7$ l'algorithme renvoie 7 et pour $n = 15$, l'algorithme renvoie 3.
- 2 Donner une implémentation en C de cet algorithme sous la forme d'une fonction dont on précisera soigneusement la spécification.

Exercice

- 1 Écrire un algorithme qui prend en entrée un entier $n > 1$ et renvoie le premier diviseur strictement supérieur à 1 de cet entier. Par exemple pour $n = 7$ l'algorithme renvoie 7 et pour $n = 15$, l'algorithme renvoie 3.
- 2 Donner une implémentation en C de cet algorithme sous la forme d'une fonction dont on précisera soigneusement la spécification.
- 3 Prouver la terminaison de cet algorithme.

Exemple introductif

Algorithme : Multiplier sans utiliser *

Entrées : $n \in \mathbb{N}, m \in \mathbb{N}$

Sorties : nm

```
1  $r \leftarrow 0$ 
2 tant que  $m > 0$  faire
3    $m \leftarrow m - 1$ 
4    $r \leftarrow r + n$ 
5 fin
6 return  $r$ 
```

On note m_0 la valeur initiale de m et on considère la propriété suivante, notée I :
« $r = (m_0 - m)n$ ». Montrer que cette propriété est vraie :

Exemple introductif

Algorithme : Multiplier sans utiliser *

Entrées : $n \in \mathbb{N}, m \in \mathbb{N}$

Sorties : nm

```
1  $r \leftarrow 0$ 
2 tant que  $m > 0$  faire
3    $m \leftarrow m - 1$ 
4    $r \leftarrow r + n$ 
5 fin
6 return  $r$ 
```

On note m_0 la valeur initiale de m et on considère la propriété suivante, notée I :
« $r = (m_0 - m)n$ ». Montrer que cette propriété est vraie :

① avant d'entrer dans la boucle ;

Exemple introductif

Algorithme : Multiplier sans utiliser *

Entrées : $n \in \mathbb{N}, m \in \mathbb{N}$

Sorties : nm

```
1  $r \leftarrow 0$ 
2 tant que  $m > 0$  faire
3    $m \leftarrow m - 1$ 
4    $r \leftarrow r + n$ 
5 fin
6 return  $r$ 
```

On note m_0 la valeur initiale de m et on considère la propriété suivante, notée I :
« $r = (m_0 - m)n$ ». Montrer que cette propriété est vraie :

- ① avant d'entrer dans la boucle ;
- ② qu'elle reste vraie à chaque tour de boucle.

Exemple introductif

Algorithme : Multiplier sans utiliser *

Entrées : $n \in \mathbb{N}, m \in \mathbb{N}$

Sorties : nm

```
1  $r \leftarrow 0$ 
2 tant que  $m > 0$  faire
3    $m \leftarrow m - 1$ 
4    $r \leftarrow r + n$ 
5 fin
6 return  $r$ 
```

On note m_0 la valeur initiale de m et on considère la propriété suivante, notée I :
« $r = (m_0 - m)n$ ». Montrer que cette propriété est vraie :

- ① avant d'entrer dans la boucle ;
- ② qu'elle reste vraie à chaque tour de boucle.

Que peut-on en conclure ?

Définitions

- Un **invariant de boucle** est une propriété qui :

Définitions

- Un **invariant de boucle** est une propriété qui :
 - est vraie à l'entrée dans la boucle (**initialisation**),

Définitions

- Un **invariant de boucle** est une propriété qui :
 - est vraie à l'entrée dans la boucle (**initialisation**),
 - reste vraie à chaque itération si elle l'était à l'itération précédente (**conservation**).

Définitions

- Un **invariant de boucle** est une propriété qui :
 - est vraie à l'entrée dans la boucle (**initialisation**),
 - reste vraie à chaque itération si elle l'était à l'itération précédente (**conservation**).
- Un algorithme est dit **partiellement correct** lorsqu'il renvoie la réponse attendue quand il se termine.

Définitions

- Un **invariant de boucle** est une propriété qui :
 - est vraie à l'entrée dans la boucle (**initialisation**),
 - reste vraie à chaque itération si elle l'était à l'itération précédente (**conservation**).
- Un algorithme est dit **partiellement correct** lorsqu'il renvoie la réponse attendue quand il se termine.
- Un algorithme est dit **totalement correct** lorsqu'il est partiellement correct et que sa terminaison est prouvée.

Définitions

- Un **invariant de boucle** est une propriété qui :
 - est vraie à l'entrée dans la boucle (**initialisation**),
 - reste vraie à chaque itération si elle l'était à l'itération précédente (**conservation**).
- Un algorithme est dit **partiellement correct** lorsqu'il renvoie la réponse attendue quand il se termine.
- Un algorithme est dit **totalement correct** lorsqu'il est partiellement correct et que sa terminaison est prouvée.

Définitions

- Un **invariant de boucle** est une propriété qui :
 - est vraie à l'entrée dans la boucle (**initialisation**),
 - reste vraie à chaque itération si elle l'était à l'itération précédente (**conservation**).
- Un algorithme est dit **partiellement correct** lorsqu'il renvoie la réponse attendue quand il se termine.
- Un algorithme est dit **totalement correct** lorsqu'il est partiellement correct et que sa terminaison est prouvée.

! Prouver la correction d'un algorithme

On utilise un invariant de boucle qui en sortie de boucle fournit une propriété permettant de montrer la correction.

La méthode est similaire à une récurrence mathématique.

Exemple

Algorithme : Nombre de chiffres en base 10

Entrées : $n \in \mathbb{N}$

Sorties : p , nombre de chiffres de n dans son écriture en base 10.

```
1 si  $n = 0$  alors
2   | return 1
3 fin
4  $p \leftarrow 0$ 
5 tant que  $n > 0$  faire
6   |  $p \leftarrow p + 1$ 
7   |  $n \leftarrow \lfloor \frac{n}{10} \rfloor$ 
8 fin
9 return  $p$ 
```

Prouver que cet algorithme est correct.

Correction

Idée : à chaque tour de boucle, on incrémente p et n perd un chiffre. La somme de p et du nombre de chiffres de n est donc constante. C'est l'invariant de boucle !

Correction

Idée : à chaque tour de boucle, on incrémente p et n perd un chiffre. La somme de p et du nombre de chiffres de n est donc constante. C'est l'invariant de boucle !

Le cas $n = 0$ est trivial, on note n_0 la valeur initiale de n et on suppose donc $n_0 > 0$. On considère la propriété I : « $c(n_0) = p + c(n)$ », où $c(m)$ vaut 0 si $m = 0$ et le nombre de chiffres de m en base 10 sinon.

Correction

Idée : à chaque tour de boucle, on incrémente p et n perd un chiffre. La somme de p et du nombre de chiffres de n est donc constante. C'est l'invariant de boucle !

Le cas $n = 0$ est trivial, on note n_0 la valeur initiale de n et on suppose donc $n_0 > 0$. On considère la propriété I : « $c(n_0) = p + c(n)$ », où $c(m)$ vaut 0 si $m = 0$ et le nombre de chiffres de m en base 10 sinon.

- Initialisation : I est vraie avant d'entrer dans la boucle puisque $p = 0$ et $n = n_0$.

Correction

Idée : à chaque tour de boucle, on incrémente p et n perd un chiffre. La somme de p et du nombre de chiffres de n est donc constante. C'est l'invariant de boucle !

Le cas $n = 0$ est trivial, on note n_0 la valeur initiale de n et on suppose donc $n_0 > 0$. On considère la propriété I : « $c(n_0) = p + c(n)$ », où $c(m)$ vaut 0 si $m = 0$ et le nombre de chiffres de m en base 10 sinon.

- Initialisation : I est vraie avant d'entrer dans la boucle puisque $p = 0$ et $n = n_0$.
- Conservation : on suppose I vraie au début d'un tour de boucle et on note n' (resp. p') la valeur de n (resp. p) après ce tour de boucle :

Correction

Idée : à chaque tour de boucle, on incrémente p et n perd un chiffre. La somme de p et du nombre de chiffres de n est donc constante. C'est l'invariant de boucle !

Le cas $n = 0$ est trivial, on note n_0 la valeur initiale de n et on suppose donc $n_0 > 0$. On considère la propriété I : « $c(n_0) = p + c(n)$ », où $c(m)$ vaut 0 si $m = 0$ et le nombre de chiffres de m en base 10 sinon.

- Initialisation : I est vraie avant d'entrer dans la boucle puisque $p = 0$ et $n = n_0$.
- Conservation : on suppose I vraie au début d'un tour de boucle et on note n' (resp. p') la valeur de n (resp. p) après ce tour de boucle :

$$p' + c(n') = p + 1 + c\left(\left\lfloor \frac{n}{10} \right\rfloor\right)$$

Correction

Idee : à chaque tour de boucle, on incrémente p et n perd un chiffre. La somme de p et du nombre de chiffres de n est donc constante. C'est l'invariant de boucle !

Le cas $n = 0$ est trivial, on note n_0 la valeur initiale de n et on suppose donc $n_0 > 0$. On considère la propriété I : « $c(n_0) = p + c(n)$ », où $c(m)$ vaut 0 si $m = 0$ et le nombre de chiffres de m en base 10 sinon.

- Initialisation : I est vraie avant d'entrer dans la boucle puisque $p = 0$ et $n = n_0$.
- Conservation : on suppose I vraie au début d'un tour de boucle et on note n' (resp. p') la valeur de n (resp. p) après ce tour de boucle :

$$p' + c(n') = p + 1 + c\left(\left\lfloor \frac{n}{10} \right\rfloor\right)$$

$$p' + c(n') = p + 1 + c(n) - 1$$

Correction

Idee : à chaque tour de boucle, on incrémente p et n perd un chiffre. La somme de p et du nombre de chiffres de n est donc constante. C'est l'invariant de boucle !

Le cas $n = 0$ est trivial, on note n_0 la valeur initiale de n et on suppose donc $n_0 > 0$. On considère la propriété I : « $c(n_0) = p + c(n)$ », où $c(m)$ vaut 0 si $m = 0$ et le nombre de chiffres de m en base 10 sinon.

- Initialisation : I est vraie avant d'entrer dans la boucle puisque $p = 0$ et $n = n_0$.
- Conservation : on suppose I vraie au début d'un tour de boucle et on note n' (resp. p') la valeur de n (resp. p) après ce tour de boucle :

$$p' + c(n') = p + 1 + c\left(\left\lfloor \frac{n}{10} \right\rfloor\right)$$

$$p' + c(n') = p + 1 + c(n) - 1$$

$$p' + c(n') = c(n_0), \text{ car } I \text{ est vraie au début du tour de boucle.}$$

Correction

Idee : à chaque tour de boucle, on incrémente p et n perd un chiffre. La somme de p et du nombre de chiffres de n est donc constante. C'est l'invariant de boucle !

Le cas $n = 0$ est trivial, on note n_0 la valeur initiale de n et on suppose donc $n_0 > 0$. On considère la propriété I : « $c(n_0) = p + c(n)$ », où $c(m)$ vaut 0 si $m = 0$ et le nombre de chiffres de m en base 10 sinon.

- Initialisation : I est vraie avant d'entrer dans la boucle puisque $p = 0$ et $n = n_0$.
- Conservation : on suppose I vraie au début d'un tour de boucle et on note n' (resp. p') la valeur de n (resp. p) après ce tour de boucle :
$$p' + c(n') = p + 1 + c\left(\left\lfloor \frac{n}{10} \right\rfloor\right)$$
$$p' + c(n') = p + 1 + c(n) - 1$$
$$p' + c(n') = c(n_0), \text{ car } I \text{ est vraie au début du tour de boucle.}$$

En sortie de boucle, on a $n = 0$ et donc $p = c(n_0)$ et l'algorithme est correct.

Exemple

Algorithme : Premier diviseur

Entrées : $n \in \mathbb{N}, n > 1$

Sorties : d , premier diviseur de n strictement supérieur à 1.

```
1  $d \leftarrow 2$ 
2 tant que  $\underline{n \bmod d \neq 0}$  faire
3   |  $d \leftarrow d + 1$ 
4 fin
5 return  $d$ 
```

Prouver que cet algorithme est correct.